HELBFourmiz — Rapport de projet

Cours : Programmation III : JAVA — Q3 2025-2026 (seconde session)
Étudiant : *Julien Baron*

---

1. Introduction

Ce projet consiste en l'implémentation de HELBFourmiz, une simulation de vie mettant en scène une colonie de fourmis, réalisée en Java avec la bibliothèque graphique JavaFX.

La simulation se déroule sur une grille de 20 × 20 cases. Une fourmilière, placée en position (0,0), laisse émerger progressivement des fourmis qui parcourent l'espace de jeu à la recherche de ressources. Deux catégories de fourmis existent, avec des comportements opposés : les fourmis rouges visent systématiquement la ressource la plus proche, les bleues la plus éloignée. Chaque ressource collectée est rapportée au camp et engendre une graine volante, laquelle migre vers les zones les moins fréquentées de la carte avant d'y germer en une nouvelle ressource. Ce cycle assure le renouvellement permanent des ressources et rend la simulation continue.

Deux mécanismes viennent enrichir cette boucle. D'une part, un système de traces : chaque passage d'une fourmi modifie la couleur de la case traversée, ce qui rend visible la fréquentation de l'espace et influence directement le lieu de germination des graines. D'autre part, la pierre philosophale, un élément qui, lorsqu'une fourmi la ramasse, la transforme en ressource ou la dédouble selon une probabilité dépendant de son type.

L'ensemble des fonctionnalités demandées par l'énoncé a été implémenté, y compris les 16 cheat codes et la sauvegarde de fin de simulation au format `save.jds`.

Ce rapport s'organise comme suit. La section 2 présente les fonctionnalités de l'application, d'abord d'un point de vue fonctionnel puis technique. La section 3 analyse la structure du code et justifie les choix de design logiciel retenus, schémas à l'appui. La section 4 traite de la cohérence chromatique (à ne pas faire). La section 5 expose les limites connues de l'implémentation ainsi que les pistes d'amélioration envisageables. La section 6 conclut sur le déroulement du projet et les apprentissages qui en découlent. L'annexe documente l'usage des LLM.

2. Fonctionnalités de base

2.1 L'espace de jeu et son pavage

Fonctionnellement, l'espace de jeu est une grille carrée de 20 × 20 cases visibles. Chaque case mesure 30 pixels de côté, soit une fenêtre de 600 × 600 pixels. Toutes les entités du jeu occupent exactement une case (taille 1 × 1), et, sauf exception documentée plus loin, aucune ne peut se superposer à une autre.

Le pavage initial respecte la contrainte imposée par l'énoncé : les composantes rouge et bleue de chaque case sont nulles, seule la composante verte varie, et deux cases adjacentes (en ligne comme en colonne) présentent les deux chiffres hexadécimaux du vert inversés.

Techniquement, cette alternance est obtenue par un simple damier : une case en position (i, j) reçoit la première couleur si `(i + j) % 2 == 0`, la seconde sinon. Cette formule garantit que deux voisines orthogonales portent toujours des couleurs différentes.

2.2 La fourmilière et l'émergence des fourmis

Fonctionnellement, aucune fourmi n'est présente au lancement. Elles émergent ensuite une par une, à raison d'une par tick, sur une case adjacente libre au camp. Si aucune case adjacente n'est libre, l'émergence est simplement reportée : la fourmi attend qu'une case se libère. Le nombre de fourmis pouvant sortir du camp est égal au nombre de ressources présentes au lancement, soit 6. La répartition respecte 40 % de fourmis rouges et 60 % de bleues.

Techniquement, l'émergence est pilotée par `spawnAnt()`, appelée à chaque tick. La case de sortie est choisie par `findFreeAdjacentToCamp()`, qui récupère les voisines accessibles du camp, les mélange pour varier le point d'apparition, puis retourne la première case libre. Le type est tiré par `randomType()` selon la constante `RED_RATIO (= 0.4)`.

Un point mérite attention : le plafond de 6 porte sur les fourmis sorties du camp, et non sur la population totale. Un compteur dédié, `emergedAntCount`, est donc maintenu séparément de `ants.size()`. Sans cette distinction, les fourmis ajoutées par cheat code ou par dédoublement via la pierre philosophale auraient saturé le plafond et interrompu définitivement la sortie des fourmis.

2.3 Le déplacement et le cycle de collecte

Fonctionnellement, une fourmi choisit une ressource cible, s'en rapproche d'une case par tick (horizontalement, verticalement ou en diagonale), la collecte en se plaçant dessus, puis retourne au camp pour l'y déposer. Une fourmi ne transporte qu'une seule ressource à la fois et, lors de son retour, contourne les autres ressources qu'elle croiserait.

Techniquement, chaque fourmi porte un état (`SEARCHING` ou `RETURNING`), une cible et indique si elle transporte une ressource. Le déplacement réutilise la méthode `getNextCoordinateForTarget(current, target)` fournie en exemple par le professeur : parmi les voisines accessibles, elle retient celle qui minimise la distance à la cible.

2.4 Les graines et la germination

Fonctionnellement, chaque collecte engendre une graine, qui apparaît sur la case libre la plus proche du lieu de collecte, puis migre d'une case par tick vers une zone peu fréquentée de la carte. À l'arrivée, elle disparaît et une nouvelle ressource naît à cet emplacement. La graine n'est pas une ressource et n'intéresse pas les fourmis, mais elle occupe une case.

Techniquement, une graine possède une position et une cible verrouillée, choisie dès sa création par `findLeastVisitedCoordinate()`. L'énoncé ne définissant pas la notion de « fréquentation », celle-ci a été formalisée comme la somme des deux composantes de trace de la case, `redTraces[x][y] + blueTraces[x][y]`.

Le déplacement réutilise `getNextCoordinateForTarget(...)`, sans code spécifique.

2.5 Le système de traces

Fonctionnellement, une case jamais parcourue conserve sa couleur de pavage. À chaque passage d'une fourmi, la composante correspondant à son type est augmentée d'un delta : rouge pour une fourmi rouge, bleue pour une bleue. Les traces révèlent ainsi visuellement les axes de circulation de la colonie, et conditionnent les zones de germination.

Techniquement, deux tableaux d'entiers, `redTraces` et `blueTraces`, sont maintenus en parallèle de la grille. `addTrace(ant)` incrémente la composante appropriée de `TRACE_DELTA = 12`. La couleur affichée est ensuite reconstruite à la volée par `getCellColor()`.

Le choix de stocker les traces dans des tableaux d'entiers plutôt que de manipuler directement des chaînes hexadécimales simplifie le code : l'incrément est une addition, et les remises à zéro des cheat codes `i`, `o` et `p` deviennent triviales.

2.6 La pierre philosophale

Fonctionnellement, une pierre est présente au lancement. Elle existe en trois types — jaune, orange et rouge — conférant respectivement les probabilités P = 2, 3 et 4, tirées au hasard à chaque apparition. Une fourmi qui la ramasse a une chance sur P d'être transformée en ressource, et P−1 chances sur P d'être dédoublée sur la case libre la plus proche. La pierre réapparaît ensuite ailleurs, avec un nouveau type. Une graine qui la traverse la fait simplement disparaître et réapparaître, sans autre effet.

Techniquement, cette fonctionnalité a nécessité la distinction la plus structurante du projet : celle entre occuper et traverser. La règle générale de non-superposition impose que la pierre bloque le placement d'un nouvel élément ; mais la pierre doit pouvoir être ramassée « si elle se trouve sur le trajet » d'une fourmi, ce qui suppose de pouvoir marcher dessus. Or, la pierre n'étant jamais une cible, elle aurait été écartée de toutes les cases accessibles : aucune fourmi n'aurait jamais pu s'y poser, et le code de collecte serait resté mort.

2.7 Les commandes interactives (cheat codes)

Les seize touches de l'énoncé sont implémentées :

Touche	Action	Touche	Action
a / z	Fourmi rouge / bleue à position libre aléatoire	j	Force la collecte de toutes les ressources
e / r	Ressource / graine à position libre aléatoire	k	Force le retour au camp
u	Pierre philosophale additionnelle	l	Force la recherche de ressource
w / x	Fige les fourmis rouges / bleues	i / o	Reset des traces rouges / bleues
c / v	Fige les graines / tous les éléments mobiles	p	Reset de toutes les traces
m	Fin de simulation et création de save.jds		

Techniquement, le gel est réalisé par trois booléens (`redAntsFrozen`, `blueAntsFrozen`, `seedsFrozen`) qui font sauter l'étape de déplacement: une fourmi gelée conserve sa cible et son état, et reprend exactement où elle s'était arrêtée. La `Timeline` n'est jamais modifiée.

2.8 Gestion du temps et fin de simulation

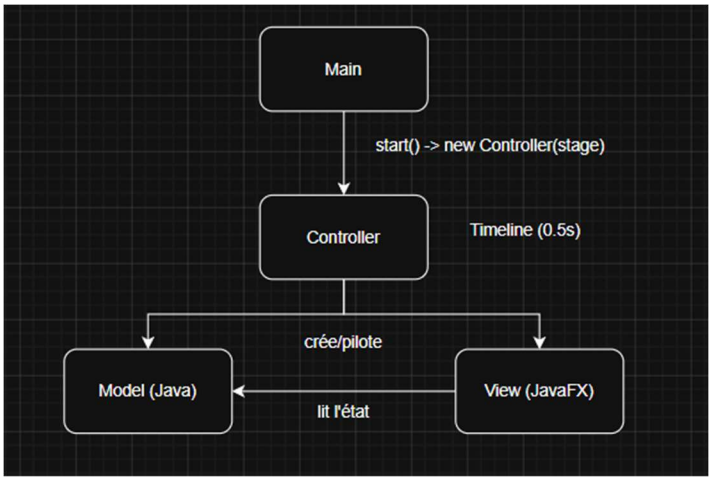
Toutes les entités agissent à la même vitesse : une action toutes les 0,5 seconde modifiable depuis le code pour accélérer la partie, conformément à l'énoncé.

La touche `m` arrête la `Timeline` et écrit `save.jds` dans le répertoire de lancement. Le fichier comporte exactement trois lignes : nombre de fourmis rouges, nombre de fourmis bleues, nombre total de ressources déposées au camp.

3. Analyse

3.1 Architecture générale : le patron MVC

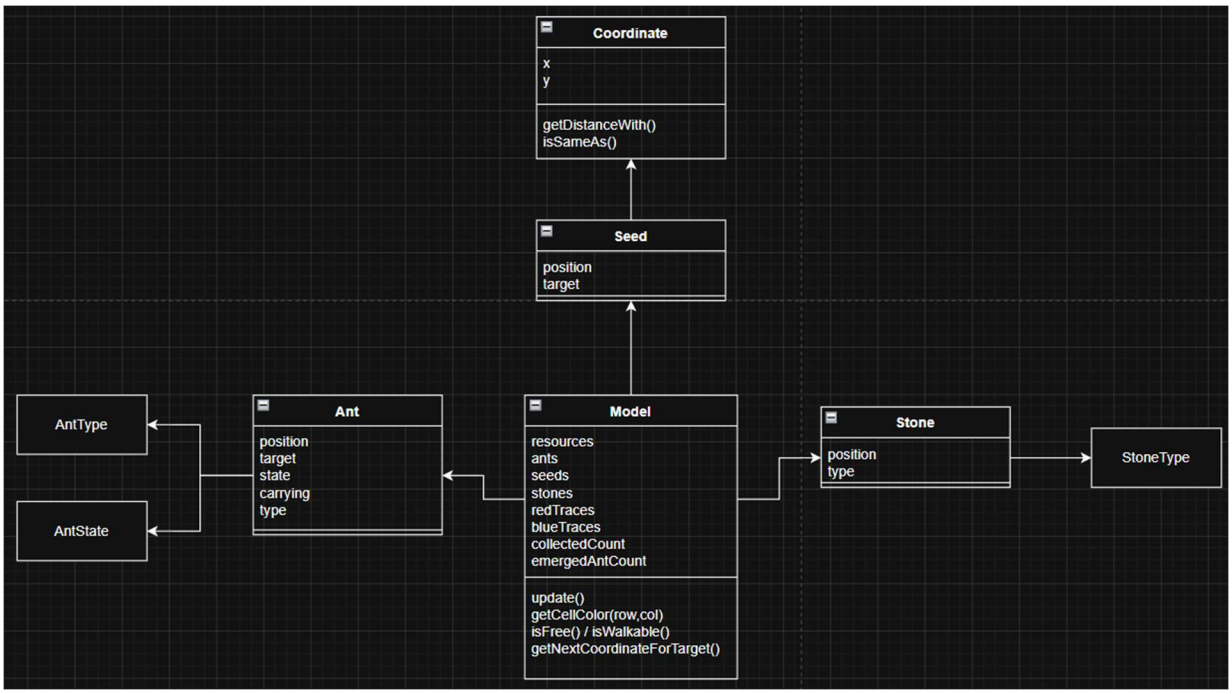
L'application suit un découpage Modèle-Vue-Contrôleur.

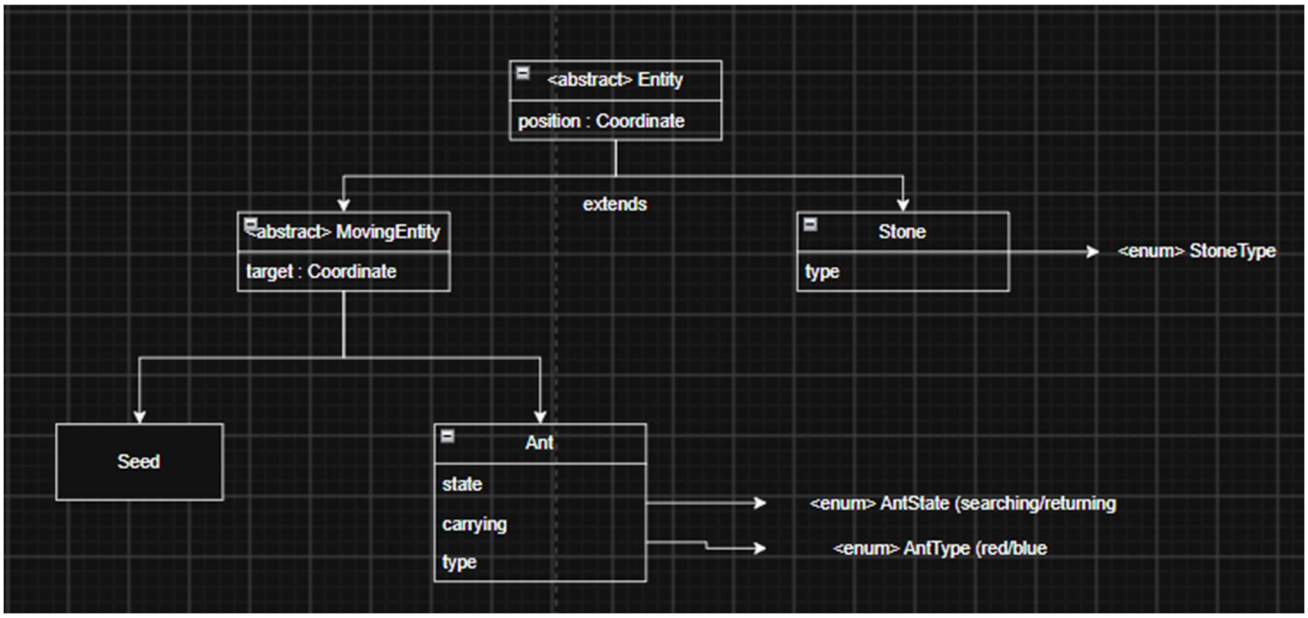


Les flèches traduisent les dépendances : le contrôleur connaît le modèle et la vue ; la vue connaît le modèle (pour lire ce qu'elle doit afficher) ; le modèle ne connaît personne.

La règle structurante est que le modèle ne contient aucun import JavaFX. Il est du Java pur : il représente les règles du jeu et l'état de la simulation, sans jamais savoir qu'il sera un jour affiché. La vue concentre tout le code graphique. Le contrôleur, quant à lui, manipule bien quelques classes JavaFX (`Timeline`, `KeyCode`, `Stage`) mais ne contient aucune logique de jeu : son `switch` clavier ne fait que traduire une touche en un appel de méthode du modèle.

3.2 Diagramme de classes





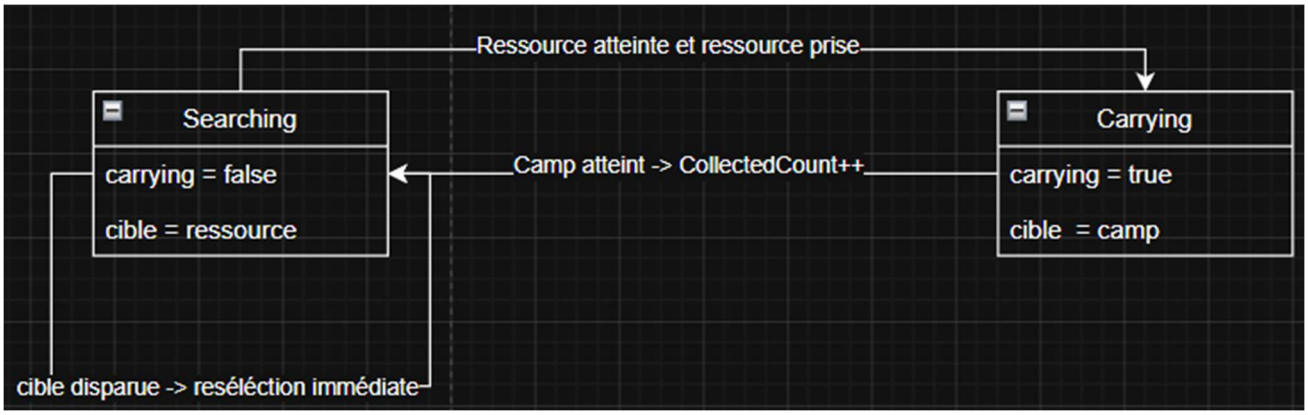
Ce diagramme montre la structure du modèle. Le Model est le conteneur central : il détient tous les éléments du jeu, le tableau des traces et le compteur, ainsi que la logique de simulation.

Les entités du jeu sont organisées en une petite hiérarchie. La classe abstraite Entity regroupe le point commun à tous les éléments : leur position. La classe abstraite MovingEntity y ajoute une target pour les éléments qui se déplacent. Ant et Seed en héritent car elles bougent vers une cible : stone, qui hérite directement d'Entity. Ces classes sont abstraites car on n'instancie jamais une 'entité'.

Ant possède en plus son état, ce qu'elle transporte et son type. Ces types sont modélisés par des énumérations (AntType, AntState, StoneType).

Enfin, Coordinate est une classe utilitaire utilisée par toutes les entités pour représenter une position et calculer les distances entre cases.

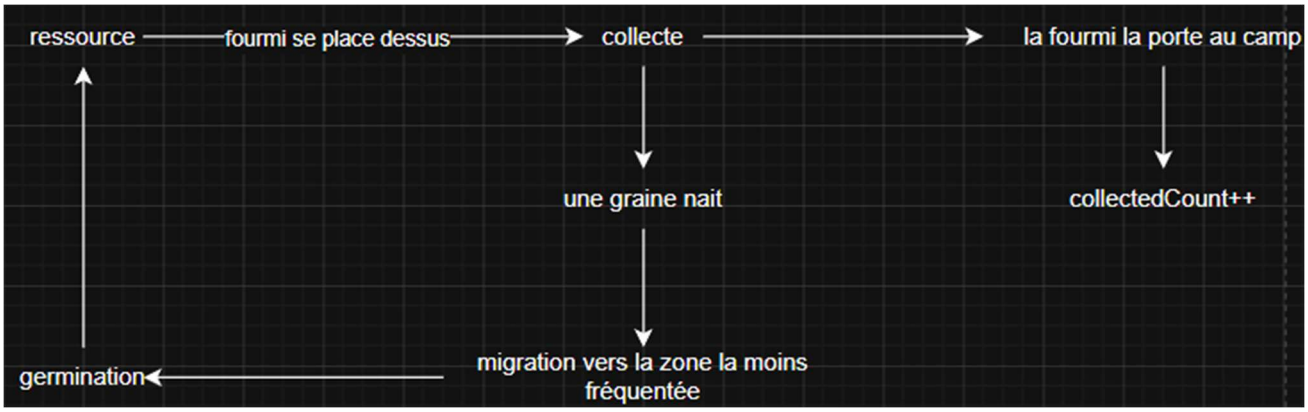
3.3 Machine à états d'une fourmi



Deux informations distinctes coexistent volontairement : `state` répond à « *où va la fourmi ?* », `carryingResource` à « *transporte-t-elle quelque chose ?* ». Les confondre serait tentant (une fourmi qui rentre au camp transporte généralement une ressource) mais les cheat codes brisent cette corrélation : k renvoie au camp des fourmis vides.

La transition « cible disparue » est également nécessaire : deux fourmis peuvent viser la même ressource. La première la ramasse ; la seconde traverserait la carte pour atteindre une case vide. `hasLostItsTargetResource()` détecte le cas et déclenche une re-sélection immédiate.

3.4 Le cycle de vie d'une ressource



Ce cycle est le moteur de la simulation : sans lui, les six ressources initiales seraient consommées puis la colonie s'immobiliserait.

4. Cohérence chromatique

Ne pas faire.

5. Limitations

5.1 Un déplacement glouton, sans contournement d'obstacle

La limitation la plus structurante tient à l'algorithme de déplacement. `getNextCoordinateForTarget()` est glouton : il choisit à chaque pas la case voisine qui minimise la distance à la cible, sans aucune vision d'ensemble. Ce n'est pas un algorithme de plus court chemin.

En conséquence, une entité peut se retrouver piégée. Si toutes les cases qui la rapprochent de sa cible sont occupées, elle ne s'éloignera jamais volontairement pour contourner l'obstacle : elle stagne, ou oscille entre

deux cases équivalentes. Le phénomène reste rare sur une grille 20 × 20 peu peuplée, mais devient probable dès que la population augmente.

5.2 Interblocages entre fourmis

Deux fourmis peuvent se bloquer mutuellement : chacune occupe la case dont l'autre a besoin. La règle de non-superposition étant absolue, aucune ne cède. La situation se résout généralement d'elle-même — les cibles changent, les ressources disparaissent.

5.3 Saturation des traces

Comme analysé en section 4, les traces plafonnent à 255, soit 21 passages. Sur une partie longue, les zones très fréquentées deviennent indistinguables et le critère de germination perd sa finesse : `findLeastVisitedCoordinate()` finit par départager des cases saturées de manière arbitraire.

5.4 Explosion démographique et saturation de la grille

Une pierre rouge (P = 4) dédouble la fourmi trois fois sur quatre. Rien ne limite la population : une succession favorable de tirages peut remplir la grille. Dans ce cas, `findRandomFreeCoordinate()` et `findNearestFreeCoordinate()` retournent `null`.

6. Conclusion

L'ensemble des fonctionnalités demandées par l'énoncé a été implémenté : le pavage et son système de traces, l'émergence des fourmis et leurs deux comportements opposés, le cycle complet de collecte et de retour au camp, les graines et leur germination dans les zones peu fréquentées, la pierre philosophale et ses trois types, les seize commandes interactives, et la sauvegarde `save.jds`. La simulation tourne en continu, sans blocage ni interruption.

Le projet a été construit étape par étape, chaque fonctionnalité étant validée avant de passer à la suivante. Cette méthode s'est révélée déterminante : elle a permis d'isoler les problèmes au moment où ils apparaissaient, plutôt que de les découvrir dans un ensemble déjà volumineux.

Le principal apprentissage tient à la nature des bugs rencontrés. Les plus coûteux n'étaient pas des erreurs de syntaxe — le compilateur les signale — mais du code qui compile et paraît correct sans produire l'effet attendu. Trois exemples résument bien cette catégorie :

- `equals()` sur la classe `Coordinate` compare des références et non des coordonnées, faute de redéfinition. Une fourmi n'aurait jamais détecté qu'elle atteignait sa cible.
- Appliquer la règle de non-superposition sans exception rend la ressource visée inatteignable : la fourmi tourne autour indéfiniment.
- Le même piège s'est reproduit avec la pierre philosophale, et n'a été résolu qu'en distinguant explicitement deux notions que je confondais : occuper une case et traverser une case.

Enfin, réutiliser les briques fournies par le cours (`Coordinate`, les méthodes d'adjacence, `getNextCoordinateForTarget`) plutôt que de repartir de zéro s'est avéré payant — à condition de les lire de manière critique. La méthode `getNextCoordinateForTarget` était correcte dans le contexte du serpent, où aucune case n'est filtrée, mais plantait dès qu'on écartait les cases occupées. Un code d'exemple est correct dans son contexte, et le transposer suppose de le comprendre d’avantage et de ne pas bêtement le coller.

Annexe : Accès LLM

Compte ChatGPT utilisé :

- Login : baronjulien5@gmail.com
- Mot de passe : HelbAdminJava

Liens conversation :

https://chatgpt.com/share/6a79b96a-5b70-83eb-9232-378e4f56a0a4

(La conversation de java q3 pour la seconde session en aout est renommée JavaQ3Aout, les autres sont les conversations pour java q4).